Yes. I would turn the challenge into a **small digital wallet + reliable money movement system**, rather than trying to build a huge banking app.

Here is the feature list I would give your team as the **implementation roadmap**, in priority order.

Complete Feature List — Serially

Phase 1 — User & Account

1. User Registration

- Name
- Email/phone/username
- Password
- Create account automatically
- Initial balance: **BDT 100,000**

2. Initial Account Funding

- Automatically credit BDT 100,000 at registration.
- Record it as an `INITIAL_CREDIT` ledger transaction.
- Never silently create money without a record.

3. Login & Authentication

- Secure password hashing
- JWT authentication
- Token expiration
- Logout/session handling

4. User Profile

- Name
- Username/phone/email
- Account status
- Account creation date

Phase 2 — Wallet / Dashboard

5. Wallet Dashboard
Show:

```
Available Balance
BDT 97,500

[ Send Money ]
[ Request Money ]

Recent Transactions
```

6. Balance Management

- Current balance
- Available balance
- Account status
- Never trust balance information supplied by the frontend.

Phase 3 — Send Money

7. Search User
Search by:

- Username
- Email
- Phone

Example:

```
Search: Bob

Bob Rahman
bob@example.com

[ Send Money ]
```

8. Send Money
User enters:

```
Receiver: Bob
Amount: BDT 2,500
Note: Lunch
```

Then:

```
[Confirm Transfer]
```

9. Transfer Validation
Reject:

- Amount ≤ 0
- Amount above available balance
- Non-existent receiver
- Suspended account
- Self-transfer
- Invalid amount format

10. Transaction Confirmation
Before sending:

```
Send BDT 2,500 to Bob?

[Cancel] [Confirm]
```

This prevents accidental transfers.

Phase 4 — The Most Important Part

This is where your project becomes more than CRUD.

11. Atomic Money Transfer

Transfer must behave as:

```
BEGIN TRANSACTION

Lock accounts
↓
Check balance
↓
Debit sender
↓
Credit receiver
↓
Create transaction
↓
Create ledger entries
↓
COMMIT
```

If anything fails:

```
ROLLBACK EVERYTHING
```

12. Concurrency Control

Handle:

```
Alice balance = 100,000

Request A → 70,000
Request B → 50,000
```
arriving simultaneously.

The system must ensure Alice cannot spend:

```
120,000
```
when she only has:

```
100,000
```
Use database transaction + row-level locking.

13. Idempotency

Protect against:

```
User taps Send twice
```
or:

```
Request succeeds
        ↓
Network timeout
        ↓
Client retries
```
The same request should **not transfer twice**.

Use an:

```
idempotency_key
```
with a unique database constraint.

14. Transaction Status

Every transfer should have a clear state:

```
PENDING
PROCESSING
COMPLETED
FAILED
CANCELLED
```
For the MVP, you can initially use:

```
PENDING
COMPLETED
FAILED
```

15. Database Rollback / Failure Handling

Demonstrate:

```
Debit succeeds
Credit fails
       ↓
ROLLBACK
       ↓
Sender gets money back
```

No partial transaction.

Phase 5 — Transaction & Ledger

16. Transaction History

Show:

```
Today

↓ Sent BDT 2,500
   To Bob

↑ Received BDT 1,200
   From Alice

↓ Sent BDT 500
   To Charlie
```

17. Transaction Details

Click a transaction:

```
Transaction ID
TX-82931

Status
COMPLETED

Amount
BDT 2,500

From
Alice

To
```

```
Bob

Date
29 Aug 2026
```

18. Double-entry-style Ledger

For every successful transfer:

```
Alice
DEBIT  2,500

Bob
CREDIT 2,500
```

This gives you an auditable financial history.

19. Ledger Immutability

Don't allow users to simply edit/delete financial records.

Instead:

```
Transaction created
        ↓
Historical record remains
```

This is important for trust and auditing.

Phase 6 — Request Money

20. Request Money

Alice:

```
Request BDT 1,200
from Bob
```

21. Incoming Money Requests

Bob sees:

```
Alice requested

BDT 1,200

[Pay] [Decline]
```

22. Accept Request

Bob presses Pay:

```
Bob → Alice
```

```
BDT 1,200
```
The same **atomic transfer engine** should be used.

Don't create a completely separate payment system.

23. Decline Request

Bob can reject it.

Status:

```
DECLINED
```
24. Request Expiration

For example:

```
Request BDT 1,200

Expires in 24 hours
```
After expiration:

```
EXPIRED
```

Phase 7 — Notifications

25. In-app Notifications

Examples:

```
You received BDT 2,500 from Bob.

Alice requested BDT 1,200 from you.

Your transfer of BDT 500 was successful.

Your transfer failed due to insufficient balance.
```
26. Notification Center

Something like:

```
Notifications
```

- `Received BDT 2,500`
- `Money request from Alice`
- `Transfer completed`

For the hackathon, **in-app notifications are enough**. Don't waste time on SMS.

Phase 8 — Security

27. Password Hashing

Never store:

```
password = 123456
```
Store a secure hash.

28. JWT Authentication

Every protected API verifies the user's identity.

29. Authorization

A user must not be able to do:

```
GET /accounts/another-user
```
and see someone else's private information.

30. Input Validation

Validate everything server-side.

Especially:

```
amount
receiver
transaction ID
request ID
```
31. Rate Limiting

Protect endpoints such as:

```
POST /login
POST /transfer
POST /request-money
```
from abuse.

For example:

```
Too many requests
        ↓
429 Too Many Requests
```

Phase 9 — Reliability

32. Network Retry Safety

Client can retry safely because of idempotency.

33. Database Constraints

Use:

- Foreign keys
- Unique constraints
- Check constraints
- NOT NULL constraints

For example:

```
amount > 0
```
should also be enforced at the database level where appropriate.

34. Transaction Audit Trail

Record:

```
Who
What
When
Amount
From
To
Status
Transaction ID
```
35. Error Handling

Don't return:

```
500 Internal Server Error
```
for everything.

Give meaningful errors:

```
Insufficient balance
Receiver not found
Transaction already processed
Account suspended
Invalid amount
```

Phase 10 — Testing

This is **very important for your specific challenge**.

36. Normal Transfer Test

```
A = 100,000
```

```
A → B = 2,500

A = 97,500
B = 102,500
```
37. Insufficient Balance Test

```
A = 1,000

A → B = 2,000

FAIL
```
Balance remains:

```
1,000
```
38. Concurrent Transfer Test

This should be one of your **main demonstrations**.

```
A = 100,000

Request 1 → 70,000
Request 2 → 50,000
```
Send them concurrently.

Expected:

```
ONE succeeds
ONE fails
```
Never:

```
A = -20,000
```
and never incorrectly allow both.

39. Duplicate Request Test

Send the same request twice:

```
idempotency_key = ABC123
```
Expected:

```
ONE transaction
```
40. Failure/Rollback Test

Force an error during the transaction.

Expected:

```
No partial balance update
```

Phase 11 — Frontend

Since you're using Flutter, keep the UI simple.

41. Splash / App Entry

42. Login Screen

43. Registration Screen

44. Home/Dashboard

45. Send Money Screen

46. Select/Search User Screen

47. Transfer Confirmation Screen

48. Transfer Result Screen

Example:

```
✓ Transfer Successful

BDT 2,500

Sent to
Bob Rahman

Transaction ID
TX-82931
```

49. Request Money Screen

50. Pending Requests Screen

51. Transaction History

52. Transaction Details

53. Notifications

54. Profile/Settings

Phase 12 — Engineering Demonstration

I would add these specifically for the judges.

55. Transaction ID / Reference

Every transaction gets something like:

```
TX-20260829-82931
```

Users can reference it when reporting an issue.

56. System Health

Optional simple admin/engineering endpoint:

```
System Status: Healthy
Database: Connected
```

Not necessary for users, but useful during demo.

57. Basic Observability

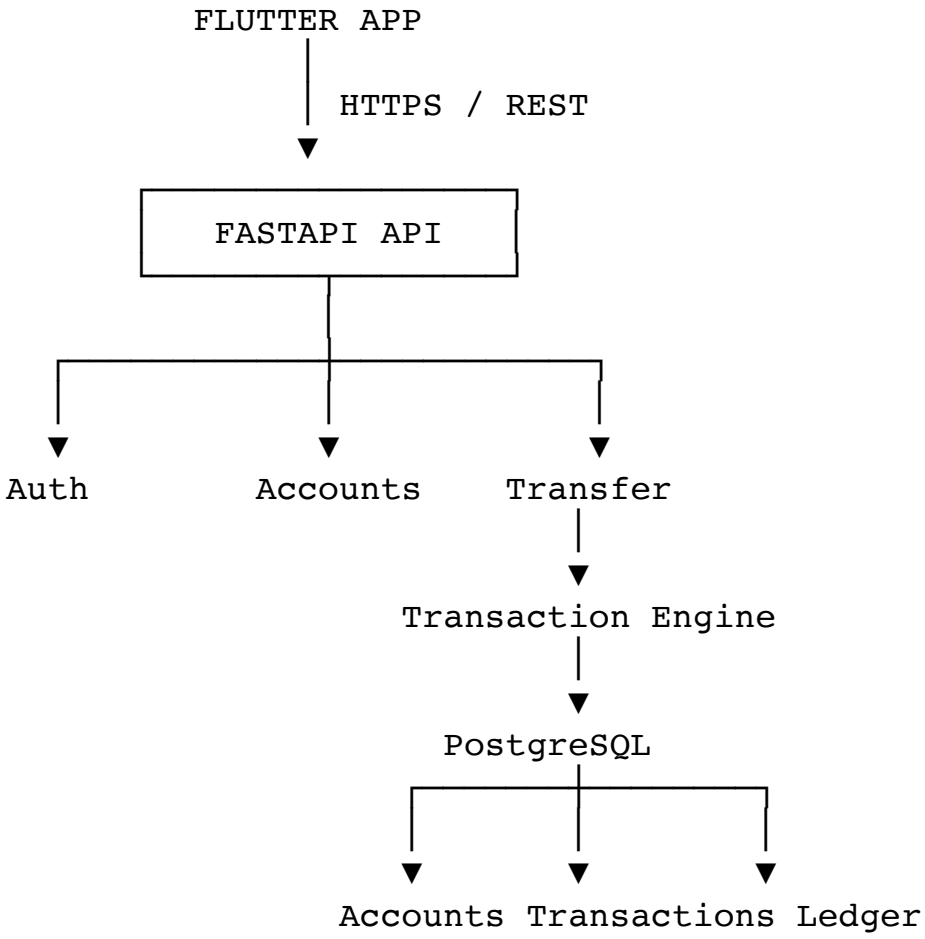
Log important events:

```
TRANSFER_STARTED
TRANSFER_COMPLETED
TRANSFER_FAILED
TRANSFER_ROLLED_BACK
```

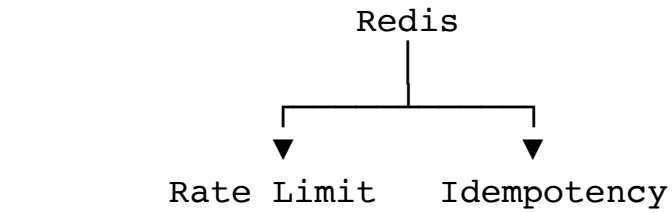
Don't log passwords, JWTs, or sensitive information.

The architecture I'd finally choose

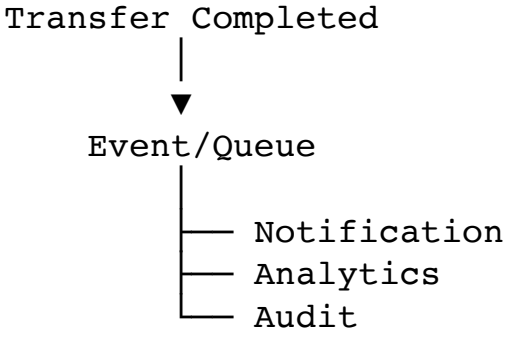
For your team, I'd keep it:



Then optionally:



And later:



But don't start with Kafka/RabbitMQ/microservices. Build the reliable transaction core first.

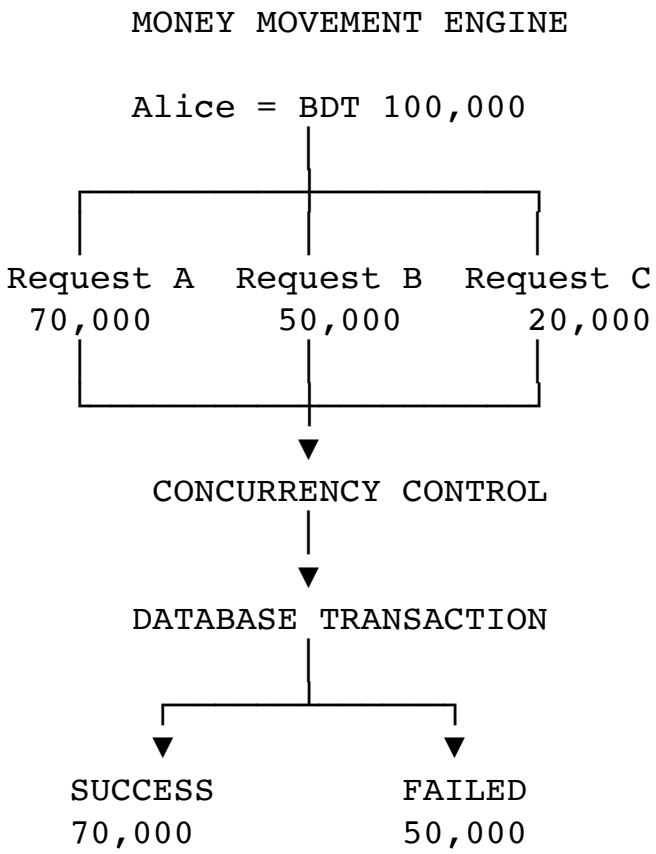
The features I would call "MUST HAVE"

If your hackathon time becomes tight, these are your **P0**:

1. User registration
2. Automatic BDT 100,000 funding
3. Login/JWT
4. Wallet/balance
5. User search
6. Send money
7. Request money
8. Accept/decline request
9. Transaction history
10. Atomic transactions
11. Row-level locking/concurrency control
12. Idempotency
13. Validation/insufficient-balance protection
14. Transaction/ledger records
15. Meaningful transaction statuses

16. Password hashing
17. Authorization
18. Rate limiting
19. Rollback/failure handling
20. **Concurrent transaction tests**

And your "wow" demo:



Then show:

`Final balance = 30,000`

and prove that the system **never created/spent money incorrectly**.

That is the part I'd make the centerpiece of the whole project.